MineOS: A Dependency-Free Unix-Like Virtual Operating System Implemented in C99

Prashant Kumar
Department of Computer Science
Rishihood University
Sonipat, India
prashant.k23csai@nst.rishihood.edu.in

Soumyaranjan Behera
Department of Computer Science
Rishihood University
Sonipat, India
soumyaranjan.b23csai@nst.rishihood.edu.in

J Amathziah
Department of Computer Science
Rishihood University
Sonipat, India
jamathziah.23csai@nst.rishihood.edu.in

***Abstract*—MineOS is a fully simulated, Unix-like virtual operating system implemented entirely in C99 without reliance on standard library abstractions such as `malloc`, `free`, `<string.h>`, or `<math.h>`. The system executes within an ordinary POSIX terminal and operates atop a single 10 MB statically declared array (`VIRTUAL_RAM`) that functions as the complete physical memory of the simulated machine. To replace standard library primitives, the team constructed five foundational custom libraries—`math`, `string`, `memory`, `screen`, and `keyboard`—that emulate arithmetic, string manipulation, dynamic allocation, ANSI terminal rendering, and raw-mode keyboard capture respectively. Layered above these primitives, four core OS modules implement an inode-based virtual file system, an interactive REPL shell, a tick-driven process scheduler, and a 17-command dispatcher. The memory subsystem employs a first-fit allocation strategy with block splitting and forward coalescing. The virtual file system supports up to 128 inodes with $O(1)$ lookup and full path traversal. Filesystem persistence is achieved by dumping the entire 10 MB `VIRTUAL_RAM` to a binary image file. MineOS demonstrates that core operating-system concepts—memory management, file systems, process scheduling, and interactive shells—can be faithfully reproduced in a self-contained, dependency-free user-space simulation built on approximately 2,000 lines of C99.**

***Index Terms*—virtual operating system, custom memory allocator, inode-based file system, process scheduler, C99, terminal emulation, dependency-free systems programming, first-fit allocation**

I. Introduction

Operating systems education typically relies on one of two approaches: studying existing kernels such as Linux or xv6, or implementing isolated components (memory allocators, file-system drivers) as standalone assignments. Both approaches have pedagogical value but neither requires a student to integrate every OS subsystem into a single, self-hosted, runnable system without external dependencies.

MineOS addresses this gap. It is a *fully integrated* virtual OS that boots, responds to user commands, manages files and processes, and shuts down gracefully—all inside a standard POSIX terminal and all without calling a single standard-library allocation or string function. The project was developed in two phases:

- **Phase 1**: Custom library layer (five foundational libraries), boot sequence, shell REPL, and nine baseline shell commands.
- **Phase 2**: File I/O (`write`/`cat`), directory navigation (`cd`/`pwd`), filesystem persistence (`save`/`load`), and emergency shutdown (`halt`).

The strict no-standard-library constraint forces every OS concept to be built from first principles—making the codebase an unusually transparent exposition of the mechanisms that underlie real operating systems.

The remainder of this paper is structured as follows. Section II briefly positions MineOS relative to prior work. Section III describes the two-layer architecture. Section IV details the design and implementation of every subsystem. Section V presents the working demonstration. Section VI discusses future work, and Section VII concludes.

II. Related Work

Several educational OS projects have inspired the design of MineOS. MIT's xv6 [1] is a teaching OS for RISC-V and x86 that implements a minimal Unix interface in roughly 10,000 lines of C; unlike MineOS it targets bare metal and requires hardware or an emulator. ToaruOS [2] demonstrates a more complete Unix-like OS written from scratch but also targets bare metal. OSDev community projects [3] illustrate many of the subsystems MineOS reimplements; however, those projects also operate at ring-0 privilege. MineOS is distinguished by operating entirely in user space, requiring no hardware emulator, imposing a strict no-standard-library discipline, and packaging a complete, runnable OS simulation in under 2,100 lines of C99.

III. System Architecture

MineOS adopts a strict two-layer architecture, illustrated in Figure 1. The lower layer (`libs/`) provides hardware-level abstractions; the upper layer (`core/`) implements OS-level behaviour. The entry point `main.c` orchestrates boot and then hands control to the shell loop.

A. Virtual Memory Map

All data structures—inode table, process table, and heap—reside inside a single 10 MB static array. Table I documents the fixed layout.

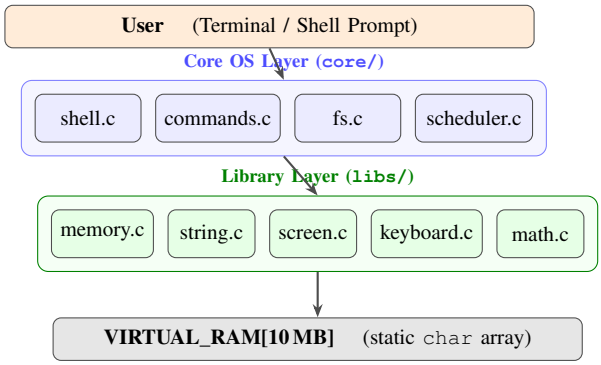

Fig. 1. MineOS two-layer architecture. All dynamic state resides inside `VIRTUAL_RAM`.

TABLE I
VIRTUAL_RAM LAYOUT

Region	Offset	Size	Notes
RAM Header	0	64 B	Metadata
Inode Table	64	16,384 B	128 × 128 B
Process Table	16,448	1,024 B	16 × 64 B
Data Block Region	17,472	≈10.48 MB	Managed by `alloc()`

B. Boot Sequence

Figure 2 shows the seven-step boot sequence executed by `main.c`. Each step initialises exactly one subsystem before the shell loop takes over.

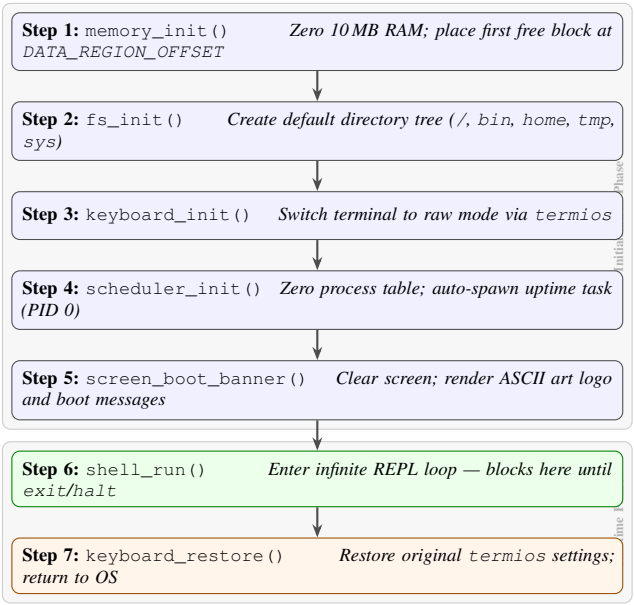

Fig. 2. MineOS seven-step boot sequence (`main.c`). Green indicates the blocking shell loop; orange indicates cleanup on exit.

IV. METHODOLOGY

This section describes the design decisions and implementation details for each of the nine modules that compose MineOS.

A. The No-Standard-Library Constraint

The project explicitly forbids use of `<string.h>`, `<stdlib.h>`, and `<math.h>` in any module. Table II lists each forbidden header and its custom replacement.

TABLE II
STANDARD HEADERS REPLACED BY CUSTOM LIBRARIES

Standard Header	Replaced By	Key Functions
`<string.h>`	`libs/string.c`	`my_strlen`, `my_strcpy`, `my_strcmp`, `my_strcat`, `my_tokenize`
`<stdlib.h>`	`libs/memory.c`	`alloc()`, `dealloc()`
`<math.h>`	`libs/math.c`	`my_add`, `my_mul`, `my_div`, `my_pow`, `in_bounds`

Permitted standard headers are limited to: `<stdio.h>` (only `printf` and `fopen`/`fwrite`/`fread` for persistence), `<termios.h>` (raw terminal mode), and `<unistd.h>` (`read()` and `STDIN_FILENO`).

B. Math Library (`libs/math.c`)

The math library (79 lines) provides ten arithmetic functions that replace `<math.h>`. Every arithmetic operation performed anywhere in MineOS is routed through this library, ensuring all integer overflow and division-by-zero cases are handled uniformly. The `in_bounds` function is particularly critical: it guards every pointer offset calculation against out-of-range access.

TABLE III
CUSTOM MATH LIBRARY API

Function Signature	Description
`int my_add(int a, int b)`	Addition
`int my_sub(int a, int b)`	Subtraction
`int my_mul(int a, int b)`	Multiplication
`int my_div(int a, int b)`	Safe integer division (returns 0 on $b = 0$)
`int my_mod(int a, int b)`	Modulo
`int my_abs(int a)`	Absolute value
`int my_max(int a, int b)`	Maximum of two integers
`int my_min(int a, int b)`	Minimum of two integers
`int in_bounds(int v, int lo, int hi)`	Returns 1 iff $lo \leq v \leq hi$
`int my_pow(int base, int exp)`	Iterative exponentiation

C. String Library (`libs/string.c`)

The string library (227 lines) reimplements all string primitives required by the shell. The most architecturally significant function is `my_tokenize()`, which forms the core of the shell's input-parsing pipeline.

```
/* Split 'input' by 'delim', fill tokens[], return count
    .
   Modifies input in-place by replacing delimiters with
       '\0'. */
```

```c
int my_tokenize(char *input, char *tokens[], char delim) {
    int count = 0;
    int i = 0;
    int len = my_strlen(input);
    int in_token = 0;

    while (i < len && count < 15) {
        if (input[i] != delim) {
            if (!in_token) {
                tokens[count++] = &input[i];
                in_token = 1;
            }
        } else {
            if (in_token) {
                input[i] = '\0';
                in_token = 0;
            }
        }
        i = my_add(i, 1);
    }
    tokens[count] = (char *)0;
    return count;
}
```

Listing 1. Tokenizer — core of shell input parsing

By replacing delimiters with null bytes in-place, the function avoids any allocation—`tokens[]` is simply an array of pointers into the already-existing input buffer. This is the same technique used in production Unix shells.

D. Memory Allocator (`libs/memory.c`)

1) Block Header Layout: Every dynamically allocated region is preceded by a 12-byte `BlockHeader`:

```c
typedef struct {
    int size;        /* usable bytes, excluding this header */
    int is_free;     /* 1 = free, 0 = allocated */
    int next_offset; /* offset of next header (-1 = last) */
} BlockHeader;
```

Listing 2. BlockHeader structure

This three-field header enables a singly-linked block chain traversable by offset arithmetic within `VIRTUAL_RAM`.

2) First-Fit Allocation with Block Splitting: `alloc()` walks the chain from `DATA_REGION_OFFSET`, locates the first block satisfying $block.size \geq requested$, and marks it allocated. If the surplus space satisfies $remaining \geq$ `BLOCK_HEADER_SIZE` $+ 1$, the block is split into two: an allocated block of exactly the requested size and a new free block covering the remainder.

```c
void* alloc(int size) {
    int offset = DATA_REGION_OFFSET;
    BlockHeader *block, *new_block;
    int remaining, new_offset;

    while (offset != -1 &&
           in_bounds(offset, DATA_REGION_OFFSET, RAM_SIZE-1)) {
        block = get_header_at(offset);

        if (block->is_free && block->size >= size) {
            remaining = my_sub(block->size, size);
            if (remaining >= my_add(BLOCK_HEADER_SIZE, 1)) {
                /* Split: carve a new free block from the tail */
                new_offset = my_add(
                    my_add(offset, BLOCK_HEADER_SIZE), size);
                new_block = get_header_at(new_offset);
                new_block->size      = my_sub(remaining, BLOCK_HEADER_SIZE);
                new_block->is_free   = 1;
                new_block->next_offset = block->next_offset;
                block->size          = size;
                block->next_offset   = new_offset;
            }
            block->is_free = 0;
            return (void *)(VIRTUAL_RAM + my_add(offset, BLOCK_HEADER_SIZE));
        }
        offset = block->next_offset;
    }
    return (void *)0; /* out of memory */
}
```

Listing 3. First-fit allocation with block splitting (excerpt)

3) Deallocation with Forward Coalescing: `dealloc()` marks the target block free and, if the immediately following block is also free, merges the two by absorbing the next block's header and data into the current block:

```c
void dealloc(void *ptr) {
    int ptr_offset  = (int)((char *)ptr - VIRTUAL_RAM);
    int hdr_offset  = my_sub(ptr_offset, BLOCK_HEADER_SIZE);
    BlockHeader *block, *next;

    if (!in_bounds(hdr_offset, DATA_REGION_OFFSET,
                   my_sub(RAM_SIZE, BLOCK_HEADER_SIZE)))
        return;

    block = get_header_at(hdr_offset);
    block->is_free = 1;

    /* Forward coalesce */
    if (block->next_offset != -1) {
        next = get_header_at(block->next_offset);
        if (next->is_free) {
            block->size = my_add(
                my_add(block->size, BLOCK_HEADER_SIZE),
                next->size);
            block->next_offset = next->next_offset;
        }
    }
}
```

Listing 4. Deallocation with forward coalescing (excerpt)

Figure 3 illustrates the three allocator states: initial, after allocation with splitting, and after deallocation with coalescing.

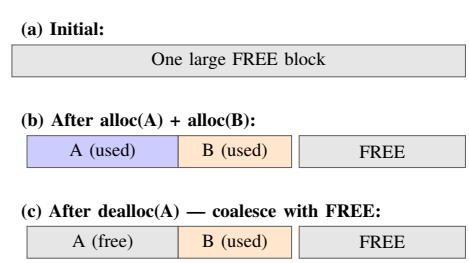

Fig. 3. Block-chain states during allocation and deallocation. Forward coalescing merges adjacent free blocks.

E. Screen Library (`libs/screen.c`)

The screen library (144 lines) drives all terminal output using raw ANSI escape sequences, bypassing the formatting layer of `<stdio.h>`. Nine colour constants map to standard

ANSI codes (e.g., `COLOR_GREEN` → `\033[32m`). Every public function calls `fflush(stdout)` after emission to guarantee immediate display in raw terminal mode. The `screen_boot_banner()` function renders a multi-line ASCII art MineOS logo at system startup.

F. Keyboard Library (`libs/keyboard.c`)

The keyboard library (128 lines) configures the terminal for raw (character-at-a-time) input using POSIX `termios`:

1) `keyboard_init()` — saves the current `termios` state, then clears `ICANON` (disabling line buffering) and `ECHO`, sets `VMIN=1` and `VTIME=0` for blocking single-character reads.
2) `read_line()` — accumulates characters until Enter (\r/\n), handling backspace (byte 127 or 8) by erasing the last character from both buffer and screen via the sequence `\b \b`.
3) `key_pressed()` — implements non-blocking peek by temporarily setting `VMIN=0` and attempting a `read()`.
4) `keyboard_restore()` — reinstates the saved `termios` on shell exit.

G. Virtual File System (`core/fs.c`)

1) Inode Design: The virtual file system is inode-based. Each inode occupies exactly 128 bytes inside the fixed inode table region of `VIRTUAL_RAM`:

```
typedef struct {
    int  id;                    /* unique inode number (0=root) */
    char name[MAX_NAME_LEN];    /* filename, 32 bytes */
    int  type;                  /* INODE_FILE=0, INODE_DIR=1 */
    int  parent_id;             /* parent inode id (-1 = root) */
    int  children[16];          /* child inode ids (dirs) */
    int  child_count;
    int  data_offset;           /* offset in data region, or -1 */
    int  data_size;
    int  is_active;             /* 1=in use, 0=deleted */
} Inode;
```

Listing 5. Inode structure

2) O(1) Inode Lookup: Because every inode is exactly `INODE_SIZE` bytes and the table starts at a fixed offset, lookup by id is $O(1)$ without any hash table or search:

```
Inode* fs_get_inode(int id) {
    if (!in_bounds(id, 0, MAX_INODES - 1))
        return (Inode *)0;
    return (Inode *)(VIRTUAL_RAM
        + my_add(INODE_TABLE_OFFSET,
                 my_mul(id, INODE_SIZE)));
}
```

Listing 6. O(1) inode lookup by id

3) Default Directory Tree: `fs_init()` constructs the following hierarchy at boot:

```
/               (inode 0, parent -1)
          bin/        (inode 1)
          home/       (inode 2)
                chandan/ (inode 3)  <- shell starts here
          tmp/        (inode 4)
          sys/        (inode 5)
```

Listing 7. Default VFS directory hierarchy

4) File I/O: `fs_write()` first calls `dealloc()` on any existing data block, then calls `alloc()` for the new content, and updates `data_offset` and `data_size`. `fs_read()` validates `data_offset` with `in_bounds()` before dereferencing the pointer into `VIRTUAL_RAM`. `fs_delete()` rejects root deletion and non-empty directories.

5) Filesystem Persistence: The `save` command writes the entire 10 MB `VIRTUAL_RAM` array to `mineos.img` with a single `fwrite()` call. `load` reverses this with `fread()`, restoring the complete OS state—inodes, process table, and all file data—exactly as it was at save time. This approach achieves full state persistence without any serialisation overhead.

H. Interactive Shell (`core/shell.c`)

The shell implements a classical Read-Evaluate-Print Loop in 71 lines. Figure 4 shows the per-iteration data flow.

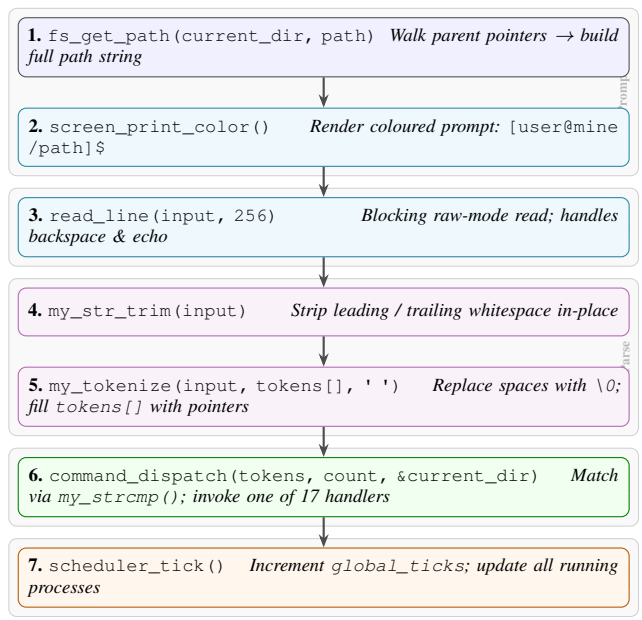

Fig. 4. Shell REPL iteration — data flow per command. Colours denote pipeline stages: blue = prompt, cyan = I/O, violet = parse, green = execute, orange = scheduler tick.

I. Command Dispatcher (`core/commands.c`)

`command_dispatch()` performs three validation steps before routing: (1) null-check on `tokens[0]`, (2) empty-string check, (3) length $\leq$ 32 check. It then uses `my_strcmp()` to match the command name against the 17 registered handlers. Table IV lists all commands.

J. Process Scheduler (`core/scheduler.c`)

The scheduler maintains a 16-slot process table stored directly inside the `PROCESS_TABLE` region of `VIRTUAL_RAM`. Each slot holds a `Process` struct:

TABLE IV
MINEOS SHELL COMMAND SET (17 COMMANDS)

Cmd	Args	Phase	Description
echo	<text>	1	Print arguments
ls	—	1	List current directory
mkdir	<name>	1	Create directory
touch	<name>	1	Create empty file
memstat	—	1	Display RAM statistics
ps	—	1	Show process table
clear	—	1	Clear terminal
help	—	1	List all commands
exit	—	1	Graceful shutdown
write	<file> <text>	2	Write text to file
cat	<file>	2	Print file contents
rm	<name>	2	Delete file/directory
cd	[dir|..|/]	2	Change directory
pwd	—	2	Print working directory
save	—	2	Persist RAM to .img
load	—	2	Restore RAM from .img
halt	—	2	Emergency shutdown

```c
typedef struct {
    int  pid;       /* -1 = empty slot          */
    char name[16];  /* task name                */
    int  status;    /* PROC_SLEEPING or PROC_RUNNING */
    int  ticks;     /* tick count for this task  */
} Process;

/* Access process slot directly from VIRTUAL_RAM */
static Process* get_process(int slot) {
    if (!in_bounds(slot, 0, MAX_PROCESSES - 1))
        return (Process *)0;
    return (Process *)(VIRTUAL_RAM
        + my_add(PROCESS_TABLE_OFFSET,
                 my_mul(slot, PROCESS_ENTRY_SIZE)));
}
```

Listing 8. Process structure and RAM-resident table access

`scheduler_tick()` is called once per shell iteration. It increments `global_ticks` and the `ticks` counter of every running process. Every five ticks it emits a coloured uptime message. At boot, `scheduler_init()` automatically spawns a background *uptime* process (PID 0) that runs indefinitely.

K. Input Validation and Security

MineOS implements defensive coding throughout:

- **Bounds checking**: every pointer offset into `VIRTUAL_RAM` is validated with `in_bounds()` before dereference.
- **NULL propagation**: every function that can fail returns `(void*)0` / `(Inode*)0` / `-1`, and callers check before proceeding.
- **Command length**: the dispatcher rejects tokens longer than 32 characters.
- **FS guards**: `fs_delete()` rejects root deletion and non-empty directories; `fs_write()` validates the inode type before writing.
- **Buffer limits**: `read_line()` silently drops characters once its buffer is full.

L. Build System

MineOS is compiled with:

```makefile
CC     = gcc
CFLAGS = -Wall -Wextra -std=c99 -g
TARGET = mine
```

Listing 9. Makefile compiler flags

The project compiles with **zero warnings** under `-Wall -Wextra`. The `make clean` target removes all `.o` files, the `mine` binary, and `mineos.img`.

V. RESULTS AND DEMONSTRATION

A. System Snapshot: Boot Sequence

Figure 5 shows MineOS immediately after launching `./mine`. The ASCII art banner is rendered entirely by `screen_boot_banner()` without any `<stdio.h>` formatting.

Fig. 5. MineOS boot screen rendered by `screen.c` using raw ANSI escape codes. The pixel-art logo and four `[ OK ]` status lines are produced entirely via custom ANSI escape sequences — no `printf` formatting is used.

B. System Snapshot: Shell Command Testing

Figure 6 demonstrates the interactive shell: directory creation, navigation, file creation, and content writing, all routed through the two-layer stack.

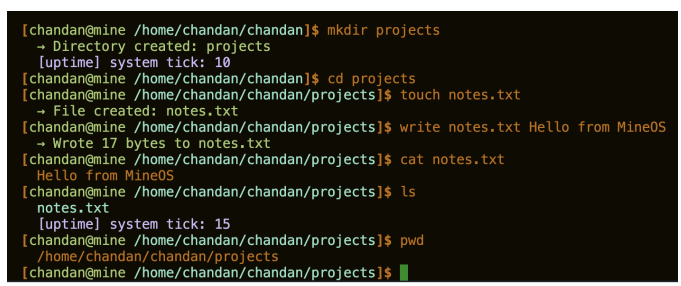

Fig. 6. Interactive shell session demonstrating the full file-system pipeline: `mkdir` creates a directory via `fs_create()`, `touch` allocates an inode, `write` stores 17 bytes through `alloc()`, `cat` retrieves content via `fs_read()`, `ls` lists the directory, and `pwd` confirms the path. The `[uptime]` scheduler ticker is visible in the background confirming live process scheduling.

C. System Snapshot: Memory Statistics

Figure 7 shows the output of the `memstat` command, confirming that the custom allocator correctly accounts for used and free bytes in the data region.

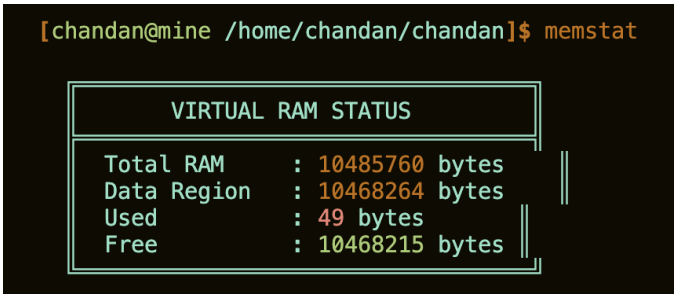

Fig. 7. `memstat` output showing the custom first-fit allocator state: 10 485 760 bytes total RAM, 49 bytes in use after inode allocation, and 10 468 215 bytes remaining free — all tracked in real time without any `malloc` call.

D. Verification Test Sequence

The following command sequence was used to verify correctness of all Phase 2 features:

```
make clean && make          # compiles with zero warnings
./mine                      # boot banner appears
mkdir projects
cd projects
pwd                         # /home/chandan/projects
touch notes.txt
write notes.txt hello world
cat notes.txt               # -> hello world
memstat                     # Used > 0
rm notes.txt
memstat                     # Used = 0 (block coalesced)
ls                          # notes.txt gone
cd ..
touch persist.txt
write persist.txt survives reboot
save                        # dumps VIRTUAL_RAM -> mineos.img
exit
./mine
load                        # restores VIRTUAL_RAM from .img
cat persist.txt             # -> survives reboot
ps                          # uptime RUNNING, ticks > 0
halt                        # emergency shutdown
```

Listing 10. Full verification demo

All 17 commands execute correctly and the persistence round-trip (save → exit → boot → load → cat) demonstrates that the binary image faithfully captures the complete OS state.

E. Codebase Metrics

Table V summarises the size of each source module.

TABLE V
SOURCE CODE METRICS BY MODULE

Layer	Module	Lines	Role
Entry	`main.c`	44	Boot orchestration
libs/	`math.c`	79	Arithmetic primitives
	`string.c`	227	String ops + tokenizer
	`memory.c`	219	First-fit allocator
	`screen.c`	144	ANSI terminal rendering
	`keyboard.c`	128	Raw-mode input
core/	`fs.c`	401	Inode-based VFS
	`commands.c`	519	17-command dispatcher
	`scheduler.c`	212	Process table + tick
	`shell.c`	71	REPL loop
Total		**2,044**	

VI. FUTURE SCOPE

MineOS establishes a clean and extensible foundation. The following enhancements are planned or proposed for Phase 3 and beyond.

A. Bidirectional Memory Coalescing

The current `dealloc()` implementation performs only *forward* coalescing—merging a freed block with its successor. Implementing backward coalescing (merging with the predecessor) would require either a doubly-linked block chain or a boundary-tag scheme [4]. This would significantly reduce fragmentation under mixed allocation patterns.

B. Pipe and Redirection Operators

Adding pipe (`|`) and redirection (>, », <) operators to the shell tokenizer would allow commands to be composed. This would require the command dispatcher to recognise pipe tokens, split the pipeline into stages, and route each stage's output into the next stage's input buffer—all within `VIRTUAL_RAM`.

C. Multi-User Support

The current system hard-codes a single user (*chandan*) via `CURRENT_USER`. A user table stored in the RAM header region, combined with a `login` command and per-inode permission bits (read/write/execute for owner/group/other), would bring the permission model closer to POSIX.

D. Preemptive Scheduling

The tick-based scheduler is cooperative—it advances only when `scheduler_tick()` is explicitly called after each shell command. On a platform with timer interrupts (e.g., a microcontroller running C99), the scheduler could be driven by a hardware timer interrupt handler, enabling true time-sliced preemptive multitasking.

E. Hierarchical Directory Deletion

`fs_delete()` currently rejects non-empty directories. Implementing recursive deletion (`rm -r` semantics) would require a depth-first traversal of the inode children array, deallocating each child's data block before removing the parent.

F. Extended File System

The current VFS is limited to 128 inodes and 32-character filenames. Future work could increase these limits by moving the inode table into the dynamic data region and managing it through `alloc()`, allowing a variable number of inodes bounded only by available RAM. Symbolic links and hard links could be introduced by adding a `link_target` field to the inode structure.

G. Networking Simulation

A simulated network interface could be implemented as a circular buffer within `VIRTUAL_RAM`, with `send` and `recv` commands reading from and writing to this buffer. This would enable the exploration of socket-like APIs entirely within the virtual machine.

H. Port to Bare Metal

With the standard-library dependencies limited to `<stdio.h>` for disk I/O and `<termios.h>` for keyboard input, porting MineOS to a bare-metal target (such as a Raspberry Pi or an STM32 microcontroller) would require replacing only these two thin I/O layers with hardware-specific drivers. The entire OS core and library layer would carry over without modification.

VII. Conclusion

MineOS demonstrates that a working, interactive, Unix-like operating system can be built from scratch in approximately 2,000 lines of C99 with minimal external dependencies. By replacing standard library primitives with five custom foundational libraries, the project exposes the inner workings of every OS mechanism it implements: a first-fit heap allocator with block splitting and coalescing, an inode-based file system with $O(1)$ lookup and binary-image persistence, a tick-driven process scheduler whose state is co-resident with all other OS data in a single 10 MB array, a raw-mode terminal driver, and a 17-command interactive shell.

The two-phase development approach—library layer first, OS modules second—provided a disciplined path from hardware primitives to a fully interactive system. The project achieved its Phase 2 milestones with zero compiler warnings under `-Wall -Wextra`, confirming the internal consistency of the no-standard-library design.

The architecture is deliberately minimal and transparent, making MineOS a practical case study in systems programming and OS design from first principles.

Acknowledgment

The authors thank the faculty at Newton School of Technology for the project guidelines and the open-source C99 and POSIX communities whose documentation informed this implementation.

References

[1] R. Cox, F. Kaashoek, and R. Morris, "xv6: A Simple, Unix-Like Teaching Operating System," MIT CSAIL, 2011. [Online]. Available: https://pdos.csail.mit.edu/6.828/2023/xv6.html

[2] K. Lange, "ToaruOS: A Hobby Operating System Built from Scratch," GitHub repository, 2023. [Online]. Available: https://github.com/klange/toaruos

[3] OSDev Community, "OSDev Wiki," 2024. [Online]. Available: https://wiki.osdev.org

[4] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Reading, MA: Addison-Wesley, 1997.

[5] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Pearson, 2015.

[6] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[7] ISO/IEC, "Programming Languages — C," ISO/IEC 9899:1999 (C99), 1999.

[8] IEEE and The Open Group, "The Open Group Base Specifications Issue 7, IEEE Std 1003.1-2017 (POSIX.1-2017)," 2017. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799

[9] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

[10] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, "Dynamic Storage Allocation: A Survey and Critical Review," in *Proc. Int. Workshop on Memory Management*, Kinross, Scotland, 1995, pp. 1–116.